Industry-Ready Mobile OTP + CAPTCHA Implementation Guide

Overview

This document explains how to implement:

1. Mobile OTP Verification during Registration
2. Mobile OTP Verification during Forgot Password
3. CAPTCHA after multiple failed login attempts

Technology Stack:

- Spring Boot Backend
- Angular 14 Frontend
- JWT Authentication
- BCrypt Password Encryption
- MySQL Database
- Fast2SMS API for OTP
- Google reCAPTCHA for CAPTCHA

The implementation follows enterprise-level authentication and security practices.

FINAL SECURITY ARCHITECTURE

```
REGISTER
--------
Mobile OTP Verification


LOGIN
-----
JWT Authentication
+
CAPTCHA after failed login attempts


FORGOT PASSWORD
---------------
```

```
Option 1 → Email Reset Link
Option 2 → Mobile OTP
```

PART 1 — MOBILE OTP IMPLEMENTATION

OBJECTIVE

We will implement:

- OTP verification during registration
- OTP verification during forgot password
- OTP expiry mechanism
- OTP validation
- Fast2SMS integration

INDUSTRY FLOW

REGISTRATION FLOW

```
User enters registration details
        ↓
User clicks Send OTP
        ↓
Backend generates OTP
        ↓
OTP sent to mobile using Fast2SMS
        ↓
User enters OTP
        ↓
Backend verifies OTP
        ↓
User account created
```

FORGOT PASSWORD FLOW

```
User selects Mobile OTP Recovery
        ↓
Enter mobile number
        ↓
Backend sends OTP
        ↓
User enters OTP
        ↓
OTP verified
        ↓
Allow password reset
```

PART 2 — DATABASE CHANGES

FILE MODIFIED

```
server/src/main/java/com/edutech/entity/User.java
```

ADD THESE FIELDS

```java
private String otp;

private LocalDateTime otpExpiry;

private boolean mobileVerified;

private int failedLoginAttempts;
```

IMPORT REQUIRED

```java
import java.time.LocalDateTime;
```

PURPOSE OF EACH FIELD

Field	Purpose
otp	Stores generated OTP
otpExpiry	Stores OTP expiry time
mobileVerified	Whether mobile number is verified
failedLoginAttempts	Counts failed login attempts

MYSQL QUERY

```
ALTER TABLE user
ADD COLUMN otp VARCHAR(10),
ADD COLUMN otp_expiry DATETIME,
ADD COLUMN mobile_verified BOOLEAN DEFAULT FALSE,
ADD COLUMN failed_login_attempts INT DEFAULT 0;
```

PART 3 — FAST2SMS SETUP

STEP 1 — CREATE ACCOUNT

Website:

https://www.fast2sms.com

STEP 2 — GET API KEY

Dashboard:

```
Dashboard → API → API Key
```

Copy API key.

STEP 3 — STORE API KEY

FILE MODIFIED

```
server/src/main/resources/application.properties
```

ADD

```
fast2sms.api.key=YOUR_API_KEY
```

PART 4 — CREATE OTP SERVICE

FILE CREATED

```
server/src/main/java/com/edutech/service/OtpService.java
```

PURPOSE

Responsible for:

- OTP generation
- OTP sending
- OTP validation
- Fast2SMS API communication

FULL IMPLEMENTATION

```
package com.edutech.service;

import com.edutech.entity.User;
import com.edutech.repository.UserRepository;
```

```java
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.*;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

import java.time.LocalDateTime;
import java.util.Random;

@Service
public class OtpService {

    @Autowired
    private UserRepository userRepository;

    @Value("${fast2sms.api.key}")
    private String apiKey;

    private final String FAST2SMS_URL =
            "https://www.fast2sms.com/dev/bulkV2";

    public String generateOtp() {

        Random random = new Random();

        int otp = 100000 + random.nextInt(900000);

        return String.valueOf(otp);
    }

    public void sendOtp(String mobileNumber) {

        User user = userRepository
                .findByContactNumber(Long.parseLong(mobileNumber))
                .orElseThrow(() ->
                        new RuntimeException("User not found"));

        String otp = generateOtp();

        user.setOtp(otp);

        user.setOtpExpiry(
                LocalDateTime.now().plusMinutes(5)
        );

        userRepository.save(user);

        RestTemplate restTemplate = new RestTemplate();
```

```
        HttpHeaders headers = new HttpHeaders();

        headers.set("authorization", apiKey);

        headers.setContentType(
                MediaType.APPLICATION_JSON
        );

        String body = "{"
                + "\"route\":\"otp\","
                + "\"message\":\"Your OTP is " + otp + "\","
                + "\"language\":\"english\","
                + "\"numbers\":\"" + mobileNumber + "\""
                + "}";

        HttpEntity<String> entity =
                new HttpEntity<>(body, headers);

        restTemplate.exchange(
                FAST2SMS_URL,
                HttpMethod.POST,
                entity,
                String.class
        );
    }

    public boolean verifyOtp(
            String mobileNumber,
            String enteredOtp
    ) {

        User user = userRepository
                .findByContactNumber(Long.parseLong(mobileNumber))
                .orElseThrow(() ->
                        new RuntimeException("User not found"));

        if (user.getOtp() == null) {
            return false;
        }

        if (user.getOtpExpiry()
                .isBefore(LocalDateTime.now())) {

            return false;
        }

        return user.getOtp().equals(enteredOtp);
```

```
    }
}
```

PART 5 — REPOSITORY CHANGES

FILE MODIFIED

```
server/src/main/java/com/edutech/repository/UserRepository.java
```

ADD

```
Optional<User> findByContactNumber(Long contactNumber);
```

PART 6 — DTO CLASSES

FILE CREATED

```
server/src/main/java/com/edutech/dto/OtpRequest.java
```

IMPLEMENTATION

```
package com.edutech.dto;

public class OtpRequest {

    private String mobileNumber;

    public String getMobileNumber() {
        return mobileNumber;
    }
```

```
    public void setMobileNumber(String mobileNumber) {
        this.mobileNumber = mobileNumber;
    }
}
```

FILE CREATED

```
server/src/main/java/com/edutech/dto/VerifyOtpRequest.java
```

IMPLEMENTATION

```
package com.edutech.dto;

public class VerifyOtpRequest {

    private String mobileNumber;

    private String otp;

    public String getMobileNumber() {
        return mobileNumber;
    }

    public void setMobileNumber(String mobileNumber) {
        this.mobileNumber = mobileNumber;
    }

    public String getOtp() {
        return otp;
    }

    public void setOtp(String otp) {
        this.otp = otp;
    }
}
```

PART 7 — AUTH CONTROLLER APIs

FILE MODIFIED

```
server/src/main/java/com/edutech/controller/AuthController.java
```

INJECT SERVICE

```
@Autowired
private OtpService otpService;
```

SEND OTP API

```
@PostMapping("/send-otp")
public ResponseEntity<?> sendOtp(
        @RequestBody OtpRequest request
) {

    otpService.sendOtp(
            request.getMobileNumber()
    );

    return ResponseEntity.ok(
            Map.of(
                    "message",
                    "OTP sent successfully"
            )
    );
}
```

VERIFY OTP API

```
@PostMapping("/verify-otp")
public ResponseEntity<?> verifyOtp(
```

```
        @RequestBody VerifyOtpRequest request
) {

    boolean verified = otpService.verifyOtp(
            request.getMobileNumber(),
            request.getOtp()
    );

    if (!verified) {

        return ResponseEntity.badRequest().body(
                Map.of(
                        "message",
                        "Invalid or expired OTP"
                )
        );
    }

    return ResponseEntity.ok(
            Map.of(
                    "message",
                    "OTP verified successfully"
            )
    );
}
```

PART 8 — SECURITY CONFIGURATION

FILE MODIFIED

```
server/src/main/java/com/edutech/config/SecurityConfig.java
```

ADD PUBLIC ACCESS

```
.antMatchers(HttpMethod.POST,
"/api/auth/send-otp")
.permitAll()

.antMatchers(HttpMethod.POST,
```

```
"/api/auth/verify-otp")
.permitAll()
```

PART 9 — ANGULAR FRONTEND IMPLEMENTATION

REGISTRATION COMPONENT

FILE MODIFIED

```
client/src/app/auth/register/register.component.ts
```

NEW FLOW

```
Fill registration form
       ↓
Click Send OTP
       ↓
OTP input appears
       ↓
Verify OTP
       ↓
Enable Register button
```

NEW VARIABLES

```
otpSent = false;
otpVerified = false;
```

SEND OTP METHOD

```
sendOtp() {

  const mobile =
    this.registerForm.value.contactNumber;

  this.authService.sendOtp(mobile)
    .subscribe({

      next: () => {
        this.otpSent = true;
      }
    });
}
```

VERIFY OTP METHOD

```
verifyOtp() {

  this.authService.verifyOtp(
      this.registerForm.value.contactNumber,
      this.registerForm.value.otp
  ).subscribe({

      next: () => {
        this.otpVerified = true;
      }
  });
}
```

REGISTER BUTTON RULE

```
<button
  [disabled]="!otpVerified"
>
Register
</button>
```

PART 10 — FORGOT PASSWORD OTP FLOW

FRONTEND FLOW

```
Forgot Password
      ↓
Choose Mobile OTP
      ↓
Enter mobile number
      ↓
Send OTP
      ↓
Verify OTP
      ↓
Open reset-password page
```

RECOMMENDED SECURITY

After OTP verification:

Generate temporary reset session token.

Do NOT directly allow reset.

PART 11 — CAPTCHA IMPLEMENTATION

OBJECTIVE

Prevent brute-force login attacks.

INDUSTRY FLOW

```
Wrong password 1 time
      ↓
Wrong password 2 times
```

```
      ↓
Wrong password 3 times
      ↓
Show CAPTCHA
      ↓
User solves CAPTCHA
      ↓
Allow login again
```

BEST CAPTCHA OPTION

Use:

```
Google reCAPTCHA v2
```

WEBSITE

https://www.google.com/recaptcha

PART 12 — FRONTEND CAPTCHA SETUP

STEP 1 — INSTALL PACKAGE

Inside Angular project:

```
npm install ng-recaptcha
```

STEP 2 — APP MODULE

FILE MODIFIED

```
client/src/app/app.module.ts
```

IMPORT

```
import { RecaptchaModule } from 'ng-recaptcha';
```

ADD IN imports

```
RecaptchaModule
```

STEP 3 — LOGIN COMPONENT HTML

FILE MODIFIED

```
client/src/app/auth/login/login.component.html
```

CAPTCHA WIDGET

```
<div *ngIf="showCaptcha">

  <re-captcha
    siteKey="YOUR_SITE_KEY"
    (resolved)="captchaResolved($event)">
  </re-captcha>
```

```
</div>
```

STEP 4 — LOGIN COMPONENT TS

VARIABLES

```
showCaptcha = false;
captchaToken = '';
failedAttempts = 0;
```

CAPTCHA METHOD

```
captchaResolved(token: string) {
  this.captchaToken = token;
}
```

LOGIN FAILURE LOGIC

```
error: () => {

  this.failedAttempts++;

  if(this.failedAttempts >= 3) {
    this.showCaptcha = true;
  }
}
```

LOGIN BUTTON RULE

```
if(this.showCaptcha && !this.captchaToken) {
```

```
  alert('Please complete CAPTCHA');

  return;
}
```

PART 13 — BACKEND CAPTCHA VALIDATION

WHY?

Frontend validation alone is insecure.

Backend must verify CAPTCHA token with Google.

FILE CREATED

```
server/src/main/java/com/edutech/service/CaptchaService.java
```

IMPLEMENTATION

```
package com.edutech.service;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

@Service
public class CaptchaService {

    @Value("${google.recaptcha.secret}")
    private String secret;

    public boolean verifyCaptcha(String captchaResponse) {

        String url =
                "https://www.google.com/recaptcha/api/siteverify"
                + "?secret=" + secret
                + "&response=" + captchaResponse;
```

```
        RestTemplate restTemplate =
                new RestTemplate();

        String response =
                restTemplate.postForObject(
                        url,
                        null,
                        String.class
                );

        return response.contains("true");
    }
}
```

---

application.properties

```
google.recaptcha.secret=YOUR_SECRET_KEY
```

PART 14 — LOGIN SECURITY LOGIC

FILE MODIFIED

```
AuthController.java
```

LOGIN FLOW

```
If failedLoginAttempts < 3
    → normal login

If failedLoginAttempts >= 3
    → CAPTCHA required
```

AFTER SUCCESSFUL LOGIN

Reset counter:

```
user.setFailedLoginAttempts(0);
```

AFTER FAILED LOGIN

Increment:

```
user.setFailedLoginAttempts(
    user.getFailedLoginAttempts() + 1
);
```

PART 15 — INDUSTRY BEST PRACTICES

OTP SECURITY

✅ OTP expiry = 5 minutes

✅ OTP should work only once

✅ OTP length = 6 digits

✅ Rate limit OTP requests

✅ Maximum OTP retries

CAPTCHA SECURITY

✅ Backend verification mandatory

✅ CAPTCHA after 3 failures

✅ Reset attempts after successful login

PASSWORD SECURITY

✅ BCrypt encryption

✅ Strong password validation

✅ Password confirmation field

JWT SECURITY

✅ Short token expiry

✅ Role-based authorization

✅ Stateless authentication

PART 16 — FINAL AUTHENTICATION ARCHITECTURE

```
REGISTRATION
------------
Mobile OTP Verification


LOGIN
-----
JWT Authentication
+
CAPTCHA after failed attempts


FORGOT PASSWORD
---------------
1. Email Reset Link
2. Mobile OTP Recovery


PASSWORD STORAGE
```

```
----------------
BCrypt Encryption
```

FINAL RESULT

Your application now supports:

✅ Mobile OTP registration verification

✅ Mobile OTP forgot password recovery

✅ Email-based password reset

✅ CAPTCHA after failed logins

✅ JWT-based authentication

✅ BCrypt encrypted passwords

✅ OTP expiry and validation

✅ Enterprise-level authentication workflow

✅ Industry-standard security architecture